Soma

Aryaman Srikant, Geon Kim, Tri Do

May 14 2023

1 Introduction

The SOMA cube puzzle is seven separate pieces that when formed together it forms a three-by-three-by-three cube. Six of the pieces have a volume of four unit cubes each, and the remaining piece has a volume of three unit cubes. The seven pieces are described as follows:

1. The "Right Angle" piece (White)
2. The "L" piece (Yellow)
3. The "T" piece (Green)
4. The "S" piece (Orange)
5. The "Right Screw" piece (Blue)
6. The "Left Screw" piece (Red)
7. The "Branch" piece (Black)

The names of the pieces are one of the ways one of the authors described each piece by looking at its shape. For the purpose of this project, we color-coded the pieces using the same convention as in the book "Winning Ways for your mathematical plays" by Berlekamp, Conway, and Guy [1] (Note that we short-handed this notation further to use just the first letter of each color.

Since Blue and Black have the same first color, we gave "B" to Black, and "L" to Blue). It has been known that there are 240 distinct solutions from [1], and so the goal of this project is the find all solutions with step-by-step visualization of forming each solution.

2 Main Idea of Solution

Backtracking:

3 Descriptions of External Library Functions

In Python, Axes3D.voxels is a function provided by the Matplotlib library, specifically within the mpl_toolkits.mplot3d module. It is used to create a 3D voxel plot, also known as a volume rendering or a 3D grid of colored cubes.

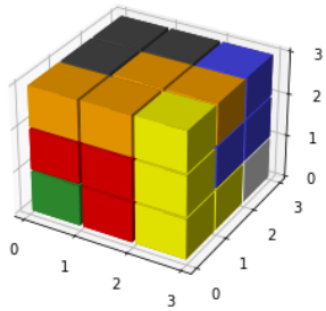
The Axes3D.voxels function takes a 3D array of boolean values or integers, where each element represents the presence or absence of a voxel at a specific location in the grid. The boolean or integer values determine the color or intensity of the voxel. For visualizing our soma cube solutions, we used a hexadecimal number system to represent different colors of pieces.

4 Algorithms and Computational Results

1. **Visualization of solutions**

   To visualize a 3D grid of a 3x3x3 cube, we utilized a NumPy 3D array to represent a solution. As shown in Figure 1, the structure of the solution array follows the shape (3, 3, 3), where the first dimension signifies the three floors of the cube. The second and third dimensions collectively represent the 9 unit cubes present on each floor.

   The current definition of the array prioritizes legibility but is not optimal for plotting purposes. Specifically, the first dimension of the array would represent the vertical floor along the yz-plane, which is not ideal for visualization. Therefore, it is necessary to transform the array into a more suitable format before proceeding with the plotting process.



```
cube = np.array([[['B','B','L'],
                  ['B','O','O'],
                  ['O','O','Y']],
                 [['B','W','L'],
                  ['R','L','L'],
                  ['R','R','Y']],
                 [['G','W','W'],
                  ['G','G','Y'],
                  ['G','R','Y']]])
```

Figure 1: Example of a solution and its corresponding 3d-array

```
def fix_orientation(solution):
    solution = solution[::-1]
    solution[0] = solution[0][::-1]
    solution[1] = solution[1][::-1]
    solution[2] = solution[2][::-1]
    solution = np.swapaxes(solution, 0,2)
    return solution
```

Figure 2: Function that modifies the order and the dimension of 3d array

Figure 2 shows the fix_orientation function that accepts a 3D array representing a solution and adjusts its orientation so that each value corresponds to the correct position within a 3x3x3 cube grid. In the original array, the values are arranged in reverse order of the indices for better readability. Therefore, the function reverses the order of the elements to restore their original sequence. Additionally, to align the first dimension with the plane perpendicular to the x-axis, the function swaps the first and third dimensions of the given array.

To assign colors to each element of the 3D array, we employed a dictionary that maps a character representing a particular piece to its corresponding hexadecimal color value as shown in Figure 3. By utilizing this dictionary, we were able to accurately allocate the appropriate color to each entry in the array before passing it into the voxels function. Consequently, when the function plotted the grid, it colored the cubes with their designated hexadecimal values.

To generate a step-by-step visualization plot, it is crucial to determine

```
# dictionary of colors by pieces
colors = {'W' : '#ffffff', 'Y' : '#ffff00' , 'G' : '#329932' , 'O' : '#ffa500',
          'L' : '#4c4cff', 'R' : '#e50000', 'B' : '#4c4c4c', 'None' : '#00000000'}
```

Figure 3: Dictionary that maps characters to hexadecimals

the appropriate order in which the pieces should be placed. The order must ensure that each piece is supported by other pieces below it. To achieve this, we implemented a function that employs a simple greedy algorithm, as shown in Figure 4. The algorithm iterates over all entries on the bottom-most floor and selects the order based on decreasing frequency. It then proceeds to move up to the next floor, repeating the process. This algorithm allows us to establish a suitable order for placing the pieces, ensuring structural integrity throughout the visualization.

```
from collections import Counter

def place_order(solution):
    order = []
    for layer in solution[::-1]:
        frequencies = Counter(layer.flatten())
        piece_sorted_by_freq = [elem for elem, freq in frequencies.most_common()]
        for piece in piece_sorted_by_freq:
            if piece not in order:
                order.append(piece)
    return order
```

Figure 4: Function that returns the order of pieces

By combining all the aforementioned helper functions, we can create a single blackbox function that encompasses all the necessary steps for generating a step-by-step visualization. Figure 5 presents this comprehensive function that handles the plotting of all the intermediate steps. Consequently, Figure 6 shows the resulting visualization obtained from executing this function.

```python
def plot_steps(solution):
    # get the placing order
    order = place_order(solution)

    # modify cube orientation
    solution = fix_orientation(solution)

    # setups for voxels function
    filled = np.ones(solution.shape)
    filled_2 = expand(filled)
    x, y, z = np.indices(np.array(filled_2.shape) + 1).astype(float) // 2
    x[0::2, :, :] += 0.03
    y[:, 0::2, :] += 0.03
    z[:, :, 0::2] += 0.03
    x[1::2, :, :] += 0.97
    y[:, 1::2, :] += 0.97
    z[:, :, 1::2] += 0.97

    # canvas construction and plots
    canvas = np.where(filled, colors['None'], 1)
    all_canvas = []
    for piece in order:
        canvas = np.where(solution==piece, colors[piece], canvas)
        all_canvas.append(canvas)

    for i in range(7):
        ax = plt.figure().add_subplot(projection='3d')
        ax.voxels(x, y, z, filled_2, facecolors=expand(all_canvas[i]))
        ax.set_xticks(np.arange(0,4,1))
        ax.set_yticks(np.arange(0,4,1))
        ax.set_zticks(np.arange(0,4,1))
    return plt.show()
```

Figure 5: Function that plots a solution step-by-step

2. Main algorithm to find solutions

We have verified that there are 240 distinct solutions to the soma cube.

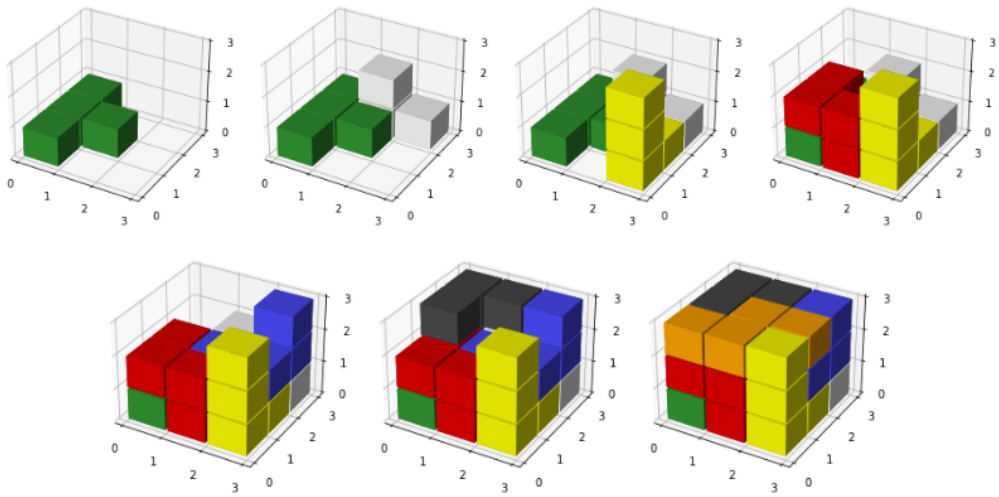


Figure 6: Example of step-by-step solution plots

5 Summary

6 Outlook

References

[1] Berlekamp, Elwyn R., Conway, John H., Guy, Richard K. (1999). Winning Ways for your mathematical plays. Volume 2: Games in particular, pg. 735. ISBN: 0-12-091102-7.

[2] Peter-Orth, C. (1984). All Solutions of the Soma Cube Puzzle. Discrete Mathematics, 57:2,105-121. DOI: https://doi.org/10.1016/0012-365X(85)90160-8.